

Service LLD

Low Level Design (LLD) for
Biomedical Knowledge Platform
for Thrombosis using Retrieval
Quality Improvement via
Reranking and Fine Tuning

System Objective and Thesis Contribution
Objective
Thesis contribution
Scope and Evolution
Version 1 Scope
Future scope
Core Architectural Principles
Embeddings: purpose, lifecycle, usage
Generator and Fine Tuned Model
Generator model
Identifiers and DOI Policy
Data Model
Document entity
Chunk entity
Chunking Strategy
Abstracts
Full text future
Vector Store Design
Qdrant configuration
Retrieval Strategy
Recall vs precision
LLM Based Reranking
Input:
Prompt:
HyDE
Control
Risk
Ask Pipeline
Risk Factor Extraction and Fine Tuning
Output structure
Confidence
Fine tuning dataset
Evaluation metrics
Citations Clarification
What citations are
What citations are not
Citation granularity
Background Ingestion without Redis
Execution
Persistence
Future Redis
API
Project Structure

System Objective and Thesis Contribution

Objective

The system enables doctors to query thrombosis related risk factors and receive structured, ranked answers grounded in scientific literature, with transparent traceability to source documents therefore the system does not aim to replace clinical decision making but to support evidence exploration.

Thesis contribution

The core contribution is **retrieval quality improvement** through:

- Vector retrieval with metadata filtering
- LLM based reranking
- Fine tuned **risk factor extraction model** producing structured outputs
- Explicit, auditable citation grounding

Retrieval-Augmented Generation (RAG) is infrastructure. The scientific contribution is **how relevance and structure improve** when reranking and fine tuning are applied.

Scope and Evolution

Version 1 Scope

- Disease: thrombosis only
- Users: doctors
- Sources: abstracts uploaded via API
- Identifier: DOI only
- Output: structured risk factors with confidence and citations
- Embeddings: single default embedding model, configurable
- Fine tuning: risk factor extraction
- Reranking: enabled
- Storage: Qdrant persistent
- Cache: in memory only
- Deployment: local, Docker

Future scope

- Full text ingestion using the same pipeline
- Larger chunking
- Section aware citations
- Redis or queue introduced without refactoring core logic
- Additional diseases

Core Architectural Principles

1. Contracts before storage
2. One embedding space per embedding model
3. Recall first, precision later
4. Structured output is mandatory
5. Citations are retrieval grounded, not bibliographic
6. Fine tuning targets extraction, not free text generation
7. Minimal infrastructure v1, extensible v2

Embeddings: purpose, lifecycle, usage

Embeddings are vector representations of text used to compute semantic similarity. They are not used for generation.

Embeddings are used in exactly three places:

1. Ingestion, embedding chunks
2. Search, embedding queries
3. Ask, embedding question or Hypothetical Document Embeddings (HyDE) text

Default embedding model:

- `nomic-embed-text`

Rationale:

- Dedicated embedding model
- Stable dimensionality
- Efficient on local hardware
- Widely validated in RAG systems

Vectors from different embedding models are never mixed. One Qdrant collection per embedding model id for example:

```
1 chunks_nomic_embed_text_dim768
```

Generator and Fine Tuned Model

Generator model

Default: `llama3.1:8b-instruct-q4_K_M`

Used for:

- HyDE generation
- Reranking
- Answer synthesis

Fine tuning target:

- Risk factor extraction

The model is trained to:

- Consume question plus context
- Output structured risk factors
- Produce stable normalized entities
- Attach evidence references

Fine tuning does **not** change retrieval.

Identifiers and DOI Policy

DOI rules

- DOI is mandatory
- PMID is rejected
- One DOI allowed once per embedding model
- Re upload rejected

Identifiers:

- `doi_original` , stored verbatim
- `doi_normalized` , lowercase trimmed
- `doc_id` , SHA256 of normalized DOI
- `chunk_id` , SHA256 of doc_id plus chunk index plus chunk text hash

Data Model

Document entity

Logical document fields, it is not stored as a separate object in Qdrant. Only Chunk entities are stored. The DOI uniqueness it is determined at the **document level**, before we write any chunks by checking the existence in QDrant for that DOI within the same embedding model space. Chunks sharing the same DOI is expected, uniqueness is about whether a document with that DOI already exists.

Unique key = `(embedding_model_id, doi_normalized)`

Fields of the Document Entity (the below info will be stored in the chunks):

- `doi_original`
- `doi_normalized`
- `doc_id`
- `disease`
- `source_type`
- `year`
- `title`
- `journal`
- `authors`
- `content_text`
- `content_hash`
- `created_at`

Chunk entity

Each document is stored as chunks. Each Chunk belongs to exactly one Document

Chunk fields:

- `chunk_id`
- `doc_id`
- `chunk_ix`
- `chunk_text`
- `char_start`
- `char_end`
- section, abstract now, later methods, results, discussion
 - Each stored text chunk includes a `section` metadata field indicating the part of the paper it originates from (e.g. `abstract` , `methods` , `results` , `discussion` , or `other`).

Chunks are always created by size, not by section; the section label is purely annotative.

In v1, all chunks are labeled as **abstract**.

When full text is ingested, sections are inferred automatically during ingestion using heuristics or structured sources.

The section metadata is used during reranking to prefer evidence from more informative parts of a paper (e.g. **results** over **discussion** or **methods**).

It may also be used for optional filtering in retrieval.

Section information is included in citations returned to the user to indicate evidence provenance.

If a section cannot be reliably determined, the value defaults to **other**.

Chunks are created by splitting the document text into fixed size segments to ensure stable embedding quality and precise retrieval.

Section boundaries do not determine chunk size, instead, each chunk is optionally annotated with the section it belongs to.

When sections are detected, multiple chunks may share the same section label, but chunking always remains size based.

- doi_original
- disease
- year
- source_type
- embedding_model_id
- created_at

Chunking Strategy

Abstracts

- If short, one chunk
- Else sentence based chunking

Defaults:

- max_chunk_tokens: 350
- overlap_tokens: 50

Full text future

- Same pipeline
- Larger chunk size
- Split section by size and annotated labels.
- Section metadata stored in payload

Vector Store Design

Qdrant configuration

- Distance: cosine
- Payload indexes:
 - disease

- year
- source_type
- doi_normalized
- section

When full text exists:

- Full text chunks weighted higher than abstract
- Implemented via reranking logic, not vector distance

Retrieval Strategy

Recall vs precision

Primary goal is recall.

Stage 1 retrieval:

- High recall vector search
- Top 30 to 50 chunks

Stage 2:

- Reranking for precision
- Top 10 chunks retained

LLM Based Reranking

Reranking corrects semantic noise from vector similarity.

Input:

- Question
- Candidate chunks with:
 - chunk_id
 - truncated text
 - section
 - DOI

Prompt:

The LLM is instructed to rank chunk ids by relevance.

Output format:

- Ordered list of chunk_ids

Validation:

- All ids must exist
- Duplicates removed
- Missing ids appended in original order
- Failure falls back to vector order

HyDE

HyDE improves recall by generating a hypothetical answer to embed.

Control

- Enabled by default
- Disabled per request

Risk

Adds latency and depends on generator quality.

Ask Pipeline

Steps:

1. Resolve effective models from config or request
2. Optional HyDE generation
3. Embed retrieval query
4. Retrieve candidates
5. Rerank
6. Select top context
7. Generate structured answer
8. Attach citations

Latency target:

- Under 50 seconds acceptable

Risk Factor Extraction and Fine Tuning

Output structure

Risk factor:

- normalized_name
- aliases
- rank
- confidence
- rationale
- evidence citations

Confidence

Relative confidence within this answer only.

Not a clinical probability.

Fine tuning dataset

Each training sample:

- question
- context chunks
- gold structured risk factors
- gold citations

Evaluation metrics

- F1 per normalized risk factor
- Citation accuracy
- Rank correlation

Citations Clarification

What citations are

Citations indicate **retrieval grounding**.

This answer statement is derived from this stored text segment.

What citations are not

They are not:

- Academic references
- Bibliographic claims
- Author curated citations

Citation granularity

- Abstract now
- Section level later for full text

Each citation includes:

- chunk_id
- doi_original
- section
- snippet

Background Ingestion without Redis

Execution

- In process background worker
- In memory job registry
- Interfaces should be used:
 - Queue interface, enqueue jobs and consume jobs.
 - enqueue(job) returns job_id
 - try_dequeue() returns job or none
 - ack(job_id) marks done
 - nack(job_id, reason) marks failed and optionally retries
 - Job state store interface, persist job metadata and status.
 - create_job(job_id, payload, state)
 - update_job(job_id, state, counts, errors)
 - get_job(job_id)
 - In memory implementations (V1)
 - InMemoryQueue

- InMemoryJobStore

Persistence

- Qdrant persists data
- Jobs state lost on restart, acceptable in v1

Future Redis

Interfaces are abstracted so Redis can be added later without changing business logic.

API

- Breaking changes allowed during thesis
- Versioned under `/v1`
- Paste the API contract in the [SwaggerEditor](#)

```

1 openapi: 3.1.0
2 info:
3   title: Biomed Knowledge Platform API
4   version: "1.0.0"
5   description: >
6     High-performance API for thrombosis knowledge extraction.
7     Supports DOI-based ingestion, Qdrant vector retrieval, and
8     RAG-based risk factor extraction.
9 servers:
10  - url: http://localhost:8000
11    description: Root server, system endpoints and versioned API
12 tags:
13  - name: System
14    description: Infrastructure and health monitoring.
15  - name: Ingestion
16    description: Operations related to DOI processing and vector
17    storage.
18  - name: Retrieval
19    description: Search and RAG (Retrieval-Augmented Generation)
20    operations.
21 paths:
22   /health:
23     get:
24       tags: [System]
25       summary: Health check
26       operationId: healthCheck
27       responses:
28         "200":
29           description: OK
30           headers:
31             X-Request-Id:
32               description: Request id for tracing
33               schema:
34                 type: string
35           content:
36             application/json:
37               schema:
38                 type: object
39                 additionalProperties: false
40                 properties:
41                   status:
42                     type: string
43                     examples: ["ok"]
44                 required: [status]
45
```

```

46 /ready:
47   get:
48     tags: [System]
49     summary: Readiness check
50     operationId: readinessCheck
51     description: >
52       Readiness indicates whether the service is ready to
receive traffic.
53       This may check downstream dependencies and configuration
presence.
54     responses:
55       "200":
56         description: Ready
57         headers:
58           X-Request-Id:
59             description: Request id for tracing
60             schema:
61               type: string
62         content:
63           application/json:
64             schema:
65               $ref: "#/components/schemas/ReadinessResponse"
66           examples:
67             ready:
68               value:
69                 status: ready
70                 checks:
71                   qdrant: ok
72                   llm: ok
73       "503":
74         description: Not ready
75         headers:
76           X-Request-Id:
77             description: Request id for tracing
78             schema:
79               type: string
80         Retry-After:
81           description: Optional backoff hint in seconds
82           schema:
83             type: integer
84         content:
85           application/json:
86             schema:
87               $ref: "#/components/schemas/ReadinessResponse"
88           examples:
89             not_ready:
90               value:
91                 status: not_ready
92                 checks:
93                   qdrant: missing_config
94                   llm: ok
95
96 /v1/ingest/items:
97   post:
98     tags: [Ingestion]
99     summary: Ingest documents (abstract now, full text later),
DOI only
100    operationId: ingestItems
101    description: >
102      Ingests one or more paper documents. DOI is mandatory and
must be unique per embedding model space.
103      Re-upload of an existing DOI is rejected.
104      Processing may run asynchronously; job state is best-
effort in v1.
105    parameters:
106      - name: X-Request-Id
107        in: header
108        required: false
109        description: Optional request id for tracing
110        schema:
111          type: string

```

```

112     - name: Idempotency-Key
113       in: header
114       required: false
115       description: Optional idempotency key for safe retries
116       schema:
117         type: string
118     requestBody:
119       required: true
120       content:
121         application/json:
122           schema:
123             $ref: "#/components/schemas/IngestBatchRequest"
124     responses:
125       "202":
126         description: Accepted
127         headers:
128           X-Request-Id:
129             description: Request id for tracing
130             schema:
131               type: string
132         content:
133           application/json:
134             schema:
135               $ref: "#/components/schemas/IngestJobAcceptedResponse"
136       "400":
137         description: Validation error or unsupported model id
138         headers:
139           X-Request-Id:
140             schema:
141               type: string
142         content:
143           application/json:
144             schema:
145               $ref: "#/components/schemas/ErrorResponse"
146       "409":
147         description: Duplicate DOI not allowed
148         headers:
149           X-Request-Id:
150             schema:
151               type: string
152         content:
153           application/json:
154             schema:
155               $ref: "#/components/schemas/ErrorResponse"
156       "429":
157         description: Too many requests
158         headers:
159           X-Request-Id:
160             schema:
161               type: string
162           Retry-After:
163             schema:
164               type: integer
165         content:
166           application/json:
167             schema:
168               $ref: "#/components/schemas/ErrorResponse"
169
170     /v1/ingest/jobs/{job_id}:
171       get:
172         tags: [Ingestion]
173         summary: Get ingestion job status
174         operationId: getIngestJobStatus
175         description: >
176           Returns best-effort job status. In v1, job state may be
177           lost after service restart.
178         parameters:
179           - name: X-Request-Id
180             in: header
181             required: false

```

```

181         description: Optional request id for tracing
182         schema:
183             type: string
184         - name: job_id
185           in: path
186           required: true
187           schema:
188               type: string
189     responses:
190         "200":
191             description: Job status
192             headers:
193                 X-Request-Id:
194                     schema:
195                         type: string
196             content:
197                 application/json:
198                     schema:
199                         $ref:
200 "#/components/schemas/IngestJobStatusResponse"
201         "404":
202             description: Job not found
203             headers:
204                 X-Request-Id:
205                     schema:
206                         type: string
207             content:
208                 application/json:
209                     schema:
210                         $ref: "#/components/schemas/ErrorResponse"
211 /v1/search:
212     post:
213         tags: [Retrieval]
214         summary: Vector search over stored chunks
215         operationId: searchChunks
216         description: >
217             Embeds the query and returns the most relevant chunks from
218             Qdrant, applying metadata filters.
219             Model ids are optional and default to rag.yaml
220             configuration.
221         parameters:
222             - name: X-Request-Id
223               in: header
224               required: false
225               description: Optional request id for tracing
226               schema:
227                   type: string
228         requestBody:
229             required: true
230             content:
231                 application/json:
232                     schema:
233                         $ref: "#/components/schemas/SearchRequest"
234     responses:
235         "200":
236             description: Search hits
237             headers:
238                 X-Request-Id:
239                     schema:
240                         type: string
241             content:
242                 application/json:
243                     schema:
244                         $ref: "#/components/schemas/SearchResponse"
245         "400":
246             description: Validation error or unsupported model id
247             headers:
248                 X-Request-Id:
249                     schema:
250                         type: string

```

```

249         content:
250         application/json:
251             schema:
252                 $ref: "#/components/schemas/ErrorResponse"
253     "429":
254         description: Too many requests
255         headers:
256             X-Request-Id:
257                 schema:
258                     type: string
259             Retry-After:
260                 schema:
261                     type: integer
262         content:
263         application/json:
264             schema:
265                 $ref: "#/components/schemas/ErrorResponse"
266
267     /v1/ask:
268         post:
269             tags: [Retrieval]
270             summary: Ask question with RAG
271             operationId: askQuestion
272             description: >
273                 Runs retrieval (optional HyDE), LLM reranking, and
274                 structured risk factor extraction.
275                 Returns an answer plus retrieval-grounded citations (chunk
276                 ids and snippets).
277                 Model ids are optional and default to rag.yaml
278                 configuration.
279                 If Prefer: respond-async is provided, the server may
280                 respond with 202 and a job id.
281             parameters:
282                 - name: X-Request-Id
283                   in: header
284                   required: false
285                   description: Optional request id for tracing
286                   schema:
287                       type: string
288                 - name: Prefer
289                   in: header
290                   required: false
291                   description: Use respond-async to request asynchronous
292                   processing
293             schema:
294                 type: string
295                 examples: ["respond-async"]
296             requestBody:
297                 required: true
298                 content:
299                     application/json:
300                         schema:
301                             $ref: "#/components/schemas/AskRequest"
302             responses:
303                 "200":
304                     description: Answer with citations
305                     headers:
306                         X-Request-Id:
307                             schema:
308                                 type: string
309                     content:
310                         application/json:
311                             schema:
312                                 $ref: "#/components/schemas/AskResponseEnvelope"
313                 "202":
314                     description: Accepted for async processing
315                     headers:
316                         X-Request-Id:
317                             schema:
318                                 type: string
319                     content:

```

```

315         application/json:
316             schema:
317                 $ref:
318 "#/components/schemas/AskJobAcceptedResponse"
319         "400":
320             description: Validation error or unsupported model id
321             headers:
322                 X-Request-Id:
323                     schema:
324                         type: string
325             content:
326                 application/json:
327                     schema:
328                         $ref: "#/components/schemas/ErrorResponse"
329         "429":
330             description: Too many requests
331             headers:
332                 X-Request-Id:
333                     schema:
334                         type: string
335                 Retry-After:
336                     schema:
337                         type: integer
338             content:
339                 application/json:
340                     schema:
341                         $ref: "#/components/schemas/ErrorResponse"
342 components:
343     securitySchemes:
344         BearerAuth:
345             type: http
346             scheme: bearer
347             bearerFormat: JWT
348             description: Optional, not enforced in v1
349     schemas:
350         ErrorResponse:
351             type: object
352             additionalProperties: false
353             properties:
354                 request_id:
355                     type: string
356                     description: Request id for tracing
357                 error:
358                     type: string
359                     description: Machine-readable error code
360                     enum: [validation_error, invalid_model_id,
361 duplicate_doi, not_found, too_many_requests]
362                 message:
363                     type: string
364                     description: Human-readable error message
365                 details:
366                     type: object
367                     description: Optional structured details
368                     additionalProperties: true
369                 required: [request_id, error, message]
370         ReadinessStatus:
371             type: string
372             enum: [ready, not_ready]
373         CheckStatus:
374             type: string
375             enum: [ok, missing_config, unhealthy, degraded, error]
376             description: >
377             Status for a single readiness check. The service should
378             pick the subset it uses.
379     ReadinessChecks:
380         type: object

```

```

383     additionalProperties: false
384     properties:
385       qdrant:
386         $ref: "#/components/schemas/CheckStatus"
387       llm:
388         $ref: "#/components/schemas/CheckStatus"
389     required: [qdrant, llm]
390
391     ReadinessResponse:
392       type: object
393       required:
394         - status
395         - checks
396       properties:
397         status:
398           $ref: "#/components/schemas/ReadinessStatus"
399         checks:
400           $ref: "#/components/schemas/ReadinessChecks"
401         errors:
402           type: object
403           additionalProperties:
404             type: object
405             additionalProperties: true
406           description: Dependency specific failure details,
present when status is not_ready.
407
408     Disease:
409       type: string
410       enum: [thrombosis]
411       description: Disease scope for v1
412
413     SourceType:
414       type: string
415       enum: [pubmed_abstract, other_abstract, full_text]
416       description: >
417         Source type. v1 supports abstracts; full_text reserved for
future pipeline expansion.
418
419     Section:
420       type: string
421       enum: [abstract, methods, results, discussion, conclusion,
other]
422       description: >
423         Section identifier for chunk provenance. v1 uses abstract;
other values reserved for full text.
424
425     IngestItem:
426       type: object
427       additionalProperties: false
428       properties:
429         doi:
430           type: string
431           description: DOI as provided by client, stored for end
users as doi_original
432           minLength: 3
433           maxLength: 512
434         disease:
435           $ref: "#/components/schemas/Disease"
436         year:
437           type: integer
438           minimum: 1800
439           maximum: 2100
440         source_type:
441           $ref: "#/components/schemas/SourceType"
442         title:
443           type: string
444           maxLength: 2000
445         journal:
446           type: string
447           maxLength: 2000
448         authors:

```

```

449         type: array
450         items:
451             type: string
452             maxLength: 512
453             maxItems: 200
454         content_text:
455             type: string
456             minLength: 1
457             maxLength: 50000
458             description: >
459                 Document text. In v1 this is typically the abstract.
In future it may contain full text.
460         required: [doi, disease, source_type, content_text]
461
462     IngestBatchRequest:
463         type: object
464         additionalProperties: false
465         properties:
466             embedding_model_id:
467                 type: string
468                 description: Optional, defaults to rag.yaml
469 default_embedding_model_id
470             examples: ["nomic-embed-text"]
471             items:
472                 type: array
473                 minItems: 1
474                 maxItems: 100
475                 items:
476                     $ref: "#/components/schemas/IngestItem"
477             required: [items]
478
479     IngestJobAcceptedResponse:
480         type: object
481         additionalProperties: false
482         properties:
483             job_id:
484                 type: string
485             state:
486                 type: string
487                 enum: [queued]
488             required: [job_id, state]
489
490     JobState:
491         type: string
492         enum: [queued, running, succeeded, failed, partial]
493
494     IngestItemState:
495         type: string
496         enum: [queued, running, succeeded, failed,
497             skipped_duplicate]
498
499     IngestItemStatus:
500         type: object
501         additionalProperties: false
502         properties:
503             doi:
504                 type: string
505             state:
506                 $ref: "#/components/schemas/IngestItemState"
507             message:
508                 type: string
509                 maxLength: 2000
510             required: [doi, state]
511
512     JobCounts:
513         type: object
514         additionalProperties: false
515         properties:
516             total:
517                 type: integer
518                 minimum: 0

```



```

517         succeeded:
518             type: integer
519             minimum: 0
520         failed:
521             type: integer
522             minimum: 0
523         skipped_duplicate:
524             type: integer
525             minimum: 0
526         required: [total, succeeded, failed, skipped_duplicate]
527
528     IngestJobStatusResponse:
529         type: object
530         additionalProperties: false
531         properties:
532             job_id:
533                 type: string
534             state:
535                 $ref: "#/components/schemas/JobState"
536             created_at:
537                 type: string
538                 format: date-time
539             updated_at:
540                 type: string
541                 format: date-time
542             effective_embedding_model_id:
543                 type: string
544                 description: Embedding model actually used
545             counts:
546                 $ref: "#/components/schemas/JobCounts"
547             items:
548                 type: array
549                 items:
550                     $ref: "#/components/schemas/IngestItemStatus"
551             required: [job_id, state, created_at, updated_at,
effective_embedding_model_id, counts, items]
552
553     SearchFilters:
554         type: object
555         additionalProperties: false
556         properties:
557             disease:
558                 $ref: "#/components/schemas/Disease"
559             year_min:
560                 type: integer
561                 minimum: 1800
562                 maximum: 2100
563             year_max:
564                 type: integer
565                 minimum: 1800
566                 maximum: 2100
567             source_type:
568                 $ref: "#/components/schemas/SourceType"
569             section:
570                 $ref: "#/components/schemas/Section"
571             required: [disease]
572
573     SearchRequest:
574         type: object
575         additionalProperties: false
576         properties:
577             query:
578                 type: string
579                 minLength: 1
580                 maxLength: 2000
581             embedding_model_id:
582                 type: string
583                 description: Optional, defaults to rag.yaml
default_embedding_model_id
584                 examples: ["nomic-embed-text"]
585             top_k:

```

```

586         type: integer
587         minimum: 1
588         maximum: 100
589         default: 20
590     cursor:
591         type: string
592         description: Optional cursor for pagination (reserved
for future use)
593     filters:
594         $ref: "#/components/schemas/SearchFilters"
595     required: [query, filters]
596
597     SearchHit:
598         type: object
599         additionalProperties: false
600         properties:
601             chunk_id:
602                 type: string
603             doc_id:
604                 type: string
605             doi:
606                 type: string
607             description: Original DOI string for end users
608             section:
609                 $ref: "#/components/schemas/Section"
610             score:
611                 type: number
612             year:
613                 type: integer
614             source_type:
615                 $ref: "#/components/schemas/SourceType"
616             title:
617                 type: string
618             chunk_text:
619                 type: string
620             description: Returned chunk text. May be truncated by
server.
621         required: [chunk_id, doc_id, doi, section, score,
chunk_text]
622
623     SearchResponse:
624         type: object
625         additionalProperties: false
626         properties:
627             effective_embedding_model_id:
628                 type: string
629             next_cursor:
630                 type: string
631             description: Optional cursor for next page (reserved for
future use)
632             hits:
633                 type: array
634                 items:
635                     $ref: "#/components/schemas/SearchHit"
636             required: [effective_embedding_model_id, hits]
637
638     RerankerMode:
639         type: string
640         enum: [llm, off]
641         description: Reranking mode
642
643     AskRequest:
644         type: object
645         additionalProperties: false
646         properties:
647             question:
648                 type: string
649                 minLength: 1
650                 maxLength: 2000
651             embedding_model_id:
652                 type: string

```

```

653         description: Optional, defaults to rag.yaml
        default_embedding_model_id
654         examples: ["nomic-embed-text"]
655         generator_model_id:
656         type: string
657         description: Optional, defaults to rag.yaml
        default_generator_model_id
658         examples: ["llama3.1:8b-instruct-q4_K_M"]
659         retrieval_top_k:
660         type: integer
661         minimum: 5
662         maximum: 100
663         default: 30
664         final_context_k:
665         type: integer
666         minimum: 3
667         maximum: 30
668         default: 10
669         hyde_enabled:
670         type: boolean
671         description: Optional, defaults to rag.yaml
        hyde_enabled_default
672         reranker_mode:
673         $ref: "#/components/schemas/RerankerMode"
674         filters:
675         $ref: "#/components/schemas/SearchFilters"
676         required: [question, filters]
677
678     AskJobAcceptedResponse:
679     type: object
680     additionalProperties: false
681     properties:
682     job_id:
683     type: string
684     state:
685     type: string
686     enum: [queued]
687     required: [job_id, state]
688
689     Citation:
690     type: object
691     additionalProperties: false
692     properties:
693     chunk_id:
694     type: string
695     doi:
696     type: string
697     description: Original DOI string for end users
698     section:
699     $ref: "#/components/schemas/Section"
700     title:
701     type: string
702     year:
703     type: integer
704     snippet:
705     type: string
706     description: Excerpt of the chunk text used as evidence
707     required: [chunk_id, doi, section, snippet]
708
709     RiskFactor:
710     type: object
711     additionalProperties: false
712     properties:
713     rank:
714     type: integer
715     minimum: 1
716     normalized_name:
717     type: string
718     description: Normalized risk factor name (controlled
719     vocabulary target)
        maxLength: 512

```

```

720     aliases:
721         type: array
722         description: Free-text aliases as extracted or implied
by the model
723     items:
724         type: string
725         maxLength: 512
726         maxItems: 50
727     confidence:
728         type: number
729         minimum: 0
730         maximum: 1
731         description: Relative confidence within this answer only
732     rationale:
733         type: string
734         description: Short explanation grounded in provided
context
735         maxLength: 5000
736     citations:
737         type: array
738         description: List of chunk_ids supporting this risk
factor
739     items:
740         type: string
741         maxItems: 20
742     required: [rank, normalized_name, aliases, confidence,
rationale, citations]
743
744     AnswerPayload:
745         type: object
746         additionalProperties: false
747         properties:
748             summary:
749                 type: string
750                 maxLength: 10000
751             risk_factors:
752                 type: array
753                 items:
754                     $ref: "#/components/schemas/RiskFactor"
755                 maxItems: 100
756             limitations:
757                 type: array
758                 items:
759                     type: string
760                     maxLength: 2000
761                     maxItems: 50
762             required: [summary, risk_factors, limitations]
763
764     AskResponseEnvelope:
765         type: object
766         additionalProperties: false
767         properties:
768             api_version:
769                 type: string
770                 examples: ["1"]
771             request_id:
772                 type: string
773             effective_embedding_model_id:
774                 type: string
775             effective_generator_model_id:
776                 type: string
777             effective_hyde_enabled:
778                 type: boolean
779             effective_reranker_mode:
780                 $ref: "#/components/schemas/RerankerMode"
781             answer:
782                 $ref: "#/components/schemas/AnswerPayload"
783             citations:
784                 type: array
785                 items:
786                     $ref: "#/components/schemas/Citation"

```

```

787         debug:
788             type: object
789             description: Optional debug information, may be disabled
in production
790             additionalProperties: true
791         required:
792             - api_version
793             - request_id
794             - effective_embedding_model_id
795             - effective_generator_model_id
796             - effective_hyde_enabled
797             - effective_reranker_mode
798             - answer
799             - citations
800
801

```

Idempotency keys exist to make **retries safe**. They prevent duplicate side effects when the client resends the *same logical request* due to network timeouts, client crashes, or uncertainty about whether the server processed the request.

What problem the idempotency key solves:

- Client sends `POST /ingest/items`
- Server processes it, but response is lost
- Client retries
- Server ingests the same DOI again
 - duplicate data, broken invariants, angry doctors

Project Structure

```

1  biomed_platform
2  |  └─ api
3  |  |  └─ routers
4  |  |  |  └─ ingest.py
5  |  |  |  └─ search.py
6  |  |  |  └─ ask.py
7  |  |  └─ app.py
8  |  └─ contracts
9  |  |  └─ ingest.py
10 |  |  └─ search.py
11 |  |  └─ ask.py
12 |  └─ providers
13 |  |  └─ embeddings.py
14 |  |  └─ generator.py
15 |  |  └─ ollama_client.py
16 |  └─ vectorstore
17 |  |  └─ qdrant_client.py
18 |  └─ rag
19 |  |  └─ chunking.py
20 |  |  └─ hyde.py
21 |  |  └─ rerank.py
22 |  |  └─ extraction.py
23 |  |  └─ pipeline.py
24 |  └─ training
25 |  |  └─ datasets.py
26 |  |  └─ train_extractor.py
27 |  |  └─ evaluate.py
28 |  └─ core
29 |  |  └─ settings.py
30 |  |  └─ logging.py

```